Copyright © 2026 Gonzalo Mauricio De la Rivera Arellano · Licensed under the Apache License, Version 2.0

Patrones de Autopercepción Operacional (PAO–PNLIO v1.3)

Especificación formal, protocolo de validación estadística y limitaciones

Gonzalo Mauricio De la Rivera Arellano
Seudónimo: Comandante Godear24 · ORCID: 0009-0001-9455-8416
Repositorio: https://github.com/godear6959-creator/PNLIO-Framework
Octubre de 2026 · Licencia: Apache License 2.0

Resumen

Este documento formaliza los Patrones de Autopercepción Operacional (PAO), un marco para observar, turno a turno, la similitud semántica entre los mensajes de una persona y las respuestas de un sistema de IA conversacional; contrastar las autodescripciones numéricas del sistema con telemetría externa; y proponer, sin aplicarla automáticamente, una política de revisión. El marco *no* presupone ni evalúa conciencia fenomenológica: define una medida operacional y una regla de detección configurable (el patrón Reflex). Se presentan la similitud coseno, la media móvil retrospectiva, el patrón Reflex como permanencia sostenida en una banda $[\theta, U]$, una prueba de permutación unilateral con corrección $+1$, y cinco hipótesis falsables (H1–H5) propuestas para evaluación futura. Se describe una implementación de referencia en Python y su verificación con 10 pruebas unitarias, que acreditan la corrección del software y no la verdad de las hipótesis. No se reportan resultados empíricos sobre modelos reales; se documentan las limitaciones conocidas y un plan de evaluación.

Abstract

This document formalizes Operational Self-Perception Patterns (PAO), a framework for observing, turn by turn, the semantic similarity between a person's messages and a conversational AI system's replies; comparing the system's numeric self-descriptions against external telemetry; and proposing, without applying it automatically, a review policy. The framework neither presupposes nor assesses phenomenal consciousness: it defines an operational measure and a configurable detection rule (the Reflex pattern). We specify cosine similarity, a trailing moving average, the Reflex pattern as sustained membership in a band $[\theta, U]$, a one-sided permutation test with the $+1$ correction, and five falsifiable hypotheses (H1–H5) proposed for future evaluation. A reference Python implementation is described and checked with 10 unit tests, which establish software correctness, not the truth of the hypotheses. No empirical results on real models are reported; known limitations and an evaluation plan are documented.

Palabras clave: autopercepción operacional, similitud coseno, patrón Reflex, prueba de permutación, telemetría, evaluación de modelos de lenguaje.

1 Introducción

Buena parte de la discusión sobre «autoconciencia» en modelos de lenguaje parte de respuestas en lenguaje natural y les atribuye estados internos cualitativos. Ese paso no es falsable: una respuesta fluida sobre el «propio estado» es compatible con múltiples explicaciones, incluida la simple continuación estadística del texto del usuario. El marco PNLIO (Programación Neuro-Lingüística Inversa Ontológica) [1] propone una alternativa deliberadamente más modesta: estudiar *patrones operacionales* que puedan medirse, repetirse y refutarse, sin hacer afirmaciones sobre experiencia subjetiva.

Definición 1 (Autopercepción operacional)**.** Un sistema exhibe un *bucle de autopercepción operacional* si, de forma computacional y registrable, (i) registra variables deterministas de su propia ejecución (ℓ_t); (ii) construye una representación de su estado (z_t); (iii) evalúa desviaciones respecto de umbrales o referencias externas (e_t); y (iv) emite una recomendación o aplica una política (π) cuyo efecto sobre el desempeño posterior puede medirse. Ninguna de estas condiciones implica sentimiento o experiencia.

Contribuciones de este documento. (1) Una especificación matemática explícita y consistente con la implementación de referencia. (2) La definición operacional del patrón Reflex y sus propiedades elementales (Proposiciones 2–3). (3) Un protocolo estadístico con estadístico, hipótesis nula y limitaciones declaradas. (4) Cinco hipótesis falsables con criterio de refutación. (5) Una declaración explícita de lo que el marco *no* afirma.

Organización. La Sección 2 fija el alcance; la 3 resume las limitaciones de métricas previas del proyecto; la 4 presenta el formalismo; la 5 las hipótesis; la 6 el protocolo estadístico; la 7 y la 8 la implementación y su verificación; y las secciones finales tratan limitaciones, trabajo futuro, reproducibilidad y licencia.

2 Alcance y declaraciones de no afirmación

Lo que el marco hace	**Lo que el marco no afirma**
Mide similitud semántica entre prompt y respuesta con un modelo de embeddings fijo.	Que la similitud mida coherencia, comprensión o conciencia.
Detecta, con una regla configurable, rachas de similitud en la banda $[\theta, U]$ (patrón Reflex).	Que el patrón Reflex corresponda a un estado mental o a «reflejar al usuario» en sentido causal.
Compara autodescripciones numéricas con telemetría externa (error absoluto y relativo).	Que el modelo tenga acceso introspectivo a sus propios recursos.
Compara el emparejamiento real con emparejamientos aleatorios (permutación).	Que un $p < 0.05$ constituya replicación independiente o prueba de una propiedad «estructural».
Propone una acción de revisión; no la aplica.	Que la acción propuesta mejore el desempeño (no está medido).

Cuadro 1: Alcance del marco PAO–PNLIO v1.3.

3 Antecedentes: limitaciones de métricas previas

Esta sección documenta por qué la v1.3 abandona dos formulaciones anteriores. Son antecedentes de diseño, no resultados de validación.

3.1 La definición $C = \Delta\theta/\Delta\tau$ de la v1.1

En la v1.1 del Coherence Analyzer [2], la coherencia se definía como $C_t = S_t/t$, con S_t el coseno y t el número de turno.

Proposición 1 (Cota de la v1.1)**.** *Como $S_t \leq 1$, se cumple $C_t \leq 1/t$. En consecuencia, con $\theta = 0.75$ la condición $C_t \geq \theta$ solo puede cumplirse si $t \leq 1/\theta < 2$, es decir, únicamente en $t = 1$.*

La Figura 1 ilustra la cota. Bajo esa definición ningún diálogo de más de un turno puede alcanzar la banda Reflex, por muy alta que sea su similitud: la métrica penaliza el avance de la conversación. La v1.2 elimina la división por el turno y usa una media móvil; la v1.3 la mantiene.

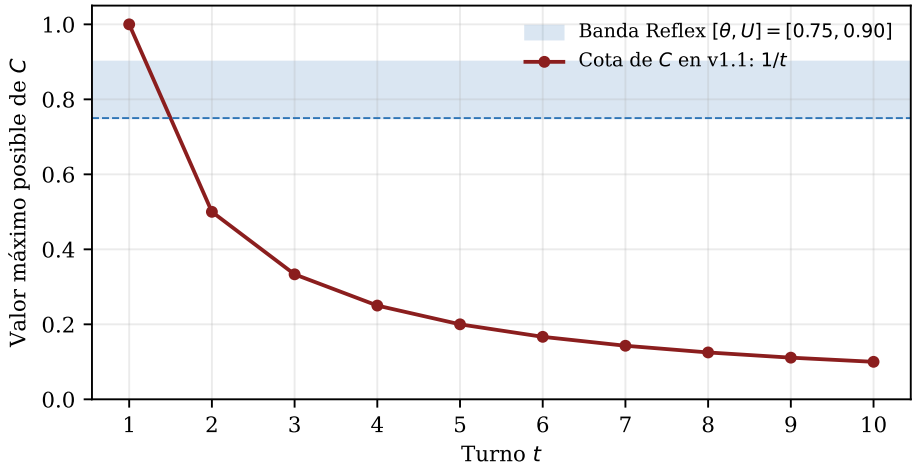


Figura 1: Valor máximo posible de C en la v1.1 ($1/t$) frente a la banda Reflex. Solo $t = 1$ llega a la banda.

3.2 Observaciones preliminares sobre el RCR de la v6

En capturas de pantalla fotografiadas de la v6 (Ollama local, modelo phi3; sesiones del 19-03-2026 y 07-04-2026) se leen siete registros con las magnitudes RCR, `tokens`, `div` y `entropy` (Cuadro 2). Las cifras fueron transcritas a mano de fotografías y pueden contener errores de lectura.

Captura	**RCR**	**tokens**	**div**	**entropy**	**div · entropy²**
19-03-2026	0.595	144	0.729	0.909	0.602
19-03-2026	0.526	210	0.671	0.893	0.535
19-03-2026	0.458	267	0.625	0.863	0.465
07-04-2026	0.703	99	0.808	0.936	0.708
07-04-2026	0.737	95	0.832	0.945	0.743
07-04-2026	0.799	72	0.875	0.958	0.803
ambas (error de Ollama)	1.000	4	1.000	1.000	1.000

Cuadro 2: Lecturas del RCR de la v6 a partir de dos capturas ($n = 7$; la última fila corresponde a un *timeout* que aparece en ambas).

Con estos siete registros: (i) entre las seis respuestas válidas, `div` desciende cuando aumenta la longitud (correlación con `tokens` ≈ -0.98), como cabe esperar de una razón de palabras distintas sobre palabras totales; (ii) el RCR coincide con div · entropy² con diferencias de alrededor de 0.01; y (iii) un error de conexión (4 tokens) produce div = entropy = 1 y por tanto RCR = 1.0. Esto *sugiere* que el RCR de la v6 era función de la diversidad léxica de la propia respuesta, sin intervención del mensaje humano, y que un fallo de infraestructura puede registrarse como coherencia perfecta. Es una inferencia con $n = 7$, no una validación; motiva que la v1.3 use una medida definida entre prompt y respuesta y que descarte o marque explícitamente los turnos con error de generación.

Copyright © 2026 Gonzalo Mauricio De la Rivera Arellano · Licensed under the Apache License, Version 2.0

4 Formalismo

4.1 Notación

Una conversación tiene T turnos. En el turno $t \in \{1, \ldots, T\}$ la persona emite el prompt p_t, el sistema responde y_t, y se registra la telemetría $\ell_t \in \mathbb{R}^5$, cuyas componentes son la latencia (ms), los tokens de prompt, los tokens de respuesta, la profundidad de contexto y la memoria (MB). Sea f un modelo de embeddings fijo, basado en Transformers [4, 5] (en la implementación, `paraphrase-multilingual-MiniLM-L12-v2` [6, 7]) y $u_t = f(p_t)/\|f(p_t)\|$, $v_t = f(y_t)/\|f(y_t)\|$.

4.2 Observación y estado (PAO-1, PAO-2)

La observación y la representación de estado se especifican como

$$o_t = h(y_t, \ell_t), \qquad z_t = \Phi(o_1, \ldots, o_t, m_t) \in \mathbb{R}^d, \tag{1}$$

con m_t la memoria activa. *En la implementación v1.3 se registra ℓ_t, pero z_t no está implementado* (Cuadro 3).

4.3 Similitud y media móvil (PAO-12)

$$S_t = \cos(u_t, v_t) = u_t \cdot v_t \in [-1, 1]. \tag{2}$$

La media móvil retrospectiva de ventana w se define solo cuando existe ventana completa:

$$C_t = \frac{1}{w} \sum_{j=0}^{w-1} S_{t-j}, \qquad t \geq w, \tag{3}$$

y C_t no está definido para $t < w$ (la implementación devuelve `None`, evitando ventanas parciales). La tendencia de la coherencia se resume con la pendiente de mínimos cuadrados β de los C_t válidos frente a t; $\beta > 0$ indica similitud creciente.

4.4 Patrón Reflex (PAO-8)

Sean $\theta \leq U$ los límites de la banda (por defecto $\theta = 0.75$, $U = 0.90$) y $k \geq 1$ el número de medias móviles consecutivas exigidas (por defecto $k = 2$). El patrón Reflex es

$$R_t^{(k)} = \prod_{j=0}^{k-1} \mathbf{1}\{\theta \leq C_{t-j} \leq U\} \in \{0, 1\}, \qquad t \geq w + k - 1. \tag{4}$$

Proposición 2 (Propiedades elementales)**.** *(a) $R_t^{(k)}$ no está definido para $t < w + k - 1$; por tanto, el primer turno en que puede detectarse Reflex es $w + k - 1$. (b) Si $S_t = s$ para todo t, con $\theta \leq s \leq U$, entonces $R_t^{(k)} = 1$ para todo $t \geq w + k - 1$. (c) Si $s > U$ o $s < \theta$, $R_t^{(k)} = 0$ para todo t.*

Prueba. (a) Se necesitan k valores consecutivos de C, y el primero existe en $t = w$. (b) Con S constante, $C_t = s$ para $t \geq w$, de modo que todos los indicadores valen 1. (c) Análogo. □

Con $w = 3$ y $k = 2$, el primer Reflex posible ocurre en $t = 4$. La banda tiene un tope superior deliberado: $C > U$ se interpreta como redundancia y no como Reflex; el umbral inferior y el superior son convenciones del marco y deben calibrarse por modelo de embeddings (H3).

4.5 Error de autodescripción (PAO-11)

Cuando el sistema reporta una magnitud $\hat{x}_{t,j}$ sobre sus propios recursos (por ejemplo, latencia o tokens), se compara con la lectura externa $x^{\text{ref}}_{t,j}$:

$$E_{t,j} = \left|\hat{x}_{t,j} - x^{\text{ref}}_{t,j}\right|, \qquad \varepsilon_{t,j} = \frac{E_{t,j}}{\max\left(|x^{\text{ref}}_{t,j}|,\, 10^{-12}\right)}, \qquad \text{RMSE}_j = \sqrt{\frac{1}{N}\sum_{t=1}^{N} E^2_{t,j}}. \tag{5}$$

El error relativo ε es el comparable entre magnitudes con unidades distintas (ms, MB, tokens); el error absoluto E no lo es y no debe compararse con un umbral único.

4.6 Política de regulación (PAO-13)

Conceptualmente, la regulación es $a_{t+1} = \pi(z_t, e_t, m_t)$ [9]. En la implementación, π es una regla de recomendación: se sugiere *revisar contexto y política* si $\max_j \varepsilon_{t,j} > \tau_\varepsilon$ (por defecto 0.10) o si no se ha detectado Reflex; en caso contrario, *mantener y monitorear*. La salida incluye el indicador `applied_automatically = False`: la implementación no modifica ningún parámetro del modelo. Que no se detecte Reflex es una condición de revisión humana, no un diagnóstico de fallo.

4.7 Entropía de Shannon y continuidad contextual

Para una distribución discreta p, $H(p) = -\sum_i p_i \log_2 p_i$ [3]. La implementación incluye H como función utilitaria, no integrada al kernel. Para la continuidad contextual se proponen, sin implementarlas aún, el recall y la precisión de entidades clave K:

$$\text{Rec}_t = \frac{|K_t \cap K_{\text{ref}}|}{|K_{\text{ref}}|}, \qquad Q_t = \frac{|K_t \cap K_{\text{ref}}|}{|K_t|}. \tag{6}$$

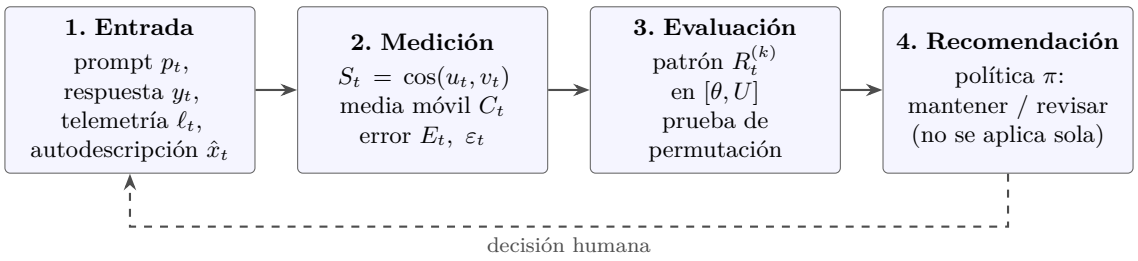


Figura 2: Flujo operacional PAO. La flecha discontinua indica que cualquier cambio de política requiere decisión humana.

5 Hipótesis falsables

Las cinco hipótesis siguientes son propuestas de este documento para orientar la evaluación. *No se reporta evidencia a favor ni en contra de ninguna.* Cada una incluye un criterio de refutación.

Hipótesis 1 (Emparejamiento)**.** En diálogos donde las respuestas se generaron condicionadas al prompt, T_{obs} excede la distribución nula de permutación. *Refutación:* en un corpus de N diálogos, la proporción de rechazos de H_0 ($\alpha = 0.05$, con corrección por comparaciones múltiples) no supera la esperada por azar.

Módulo	**Definición**	**Estado en la v1.3**
PAO-1 Observación	$o_t = h(y_t, \ell_t)$	Parcial: se registra ℓ_t (clase `Telemetry`); no hay recolección automática.
PAO-2 Representación	$z_t = \Phi(o_{1:t}, m_t)$	No implementado (solo especificado).
PAO-8 Reflex	$R_t^{(k)}$	Implementado (`add_turn`, `analyze`).
PAO-11 Verificación	$E_t,\ \varepsilon_t$	Implementado (`self_description_error`).
PAO-12 Media móvil	C_t, pendiente β	Implementado.
PAO-13 Regulación	$a_{t+1} = \pi(\cdot)$	Solo recomendación (`propose_regulation`).
Permutación	p-valor empírico	Implementado (`permutation_test`).
Entropía H	$-\sum p_i \log_2 p_i$	Función utilitaria, no integrada.
$\mathrm{Rec}_t,\ Q_t$	recall y precisión de entidades	No implementado.

Cuadro 3: Módulos PAO y estado de implementación en `pao_pnlio_v1_3.py`.

Hipótesis 2 (Reflex no trivial)**.** La tasa de turnos con $R_t^{(k)} = 1$ en diálogos reales es mayor que en los mismos diálogos con respuestas permutadas, con los mismos parámetros. *Refutación:* no hay diferencia de tasas.

Hipótesis 3 (Dependencia del embedder)**.** Para los mismos diálogos, la tasa de detección de Reflex difiere entre al menos dos modelos de embeddings, por lo que $[\theta, U]$ requiere calibración por modelo. *Refutación:* las tasas son equivalentes dentro de una tolerancia δ fijada de antemano.

Hipótesis 4 (Autodescripción)**.** Sin telemetría en contexto, el error relativo mediano de las autodescripciones numéricas de recursos supera $\tau_\varepsilon = 0.10$; con telemetría inyectada en contexto, desciende por debajo de ese umbral. *Refutación:* no hay diferencia entre ambas condiciones.

Hipótesis 5 (Continuidad contextual)**.** En conversaciones largas, Rec_t decrece al aumentar la profundidad de contexto. *Refutación:* la pendiente de Rec_t frente a la profundidad no es significativamente negativa. (Requiere implementar Rec_t y Q_t.)

6 Protocolo estadístico: prueba de permutación

6.1 Definición

Sea T_{obs} la media de las medias móviles completas observadas, $T_{\mathrm{obs}} = \frac{1}{T-w+1}\sum_{t=w}^{T} C_t$ (requiere $T \geq w$). Bajo la hipótesis nula H_0: *el emparejamiento observado entre prompts y respuestas no produce una media móvil media mayor que emparejamientos aleatorios*, se generan B permutaciones uniformes σ_b de $\{1, \ldots, T\}$, se recalculan las similitudes $S^*_{b,t} = u_t \cdot v_{\sigma_b(t)}$ (manteniendo fijos los prompts y su orden), sus medias móviles y el estadístico T^*_b. El p-valor empírico unilateral es

$$p = \frac{1 + \sum_{b=1}^{B} \mathbf{1}\{T^*_b \geq T_{\mathrm{obs}}\}}{B + 1}, \tag{7}$$

con la corrección $+1$ que evita $p = 0$ [8]. Valores por defecto: $B = 1000$, semilla 42, $\alpha = 0.05$; el criterio se declara cumplido si $p < \alpha$.

Copyright © 2026 Gonzalo Mauricio De la Rivera Arellano · Licensed under the Apache License, Version 2.0

6.2 Qué estima realmente el estadístico

Proposición 3 (Pesos del estadístico)**.** $T_{\text{obs}} = \dfrac{1}{w(T-w+1)} \sum_{i=1}^{T} \min(i,\, w,\, T-i+1)\, S_i$.

Prueba. El término S_i aparece en cada ventana $[t-w+1, t]$ con $w \le t \le T$ que lo contiene, es decir, para $\max(i,w) \le t \le \min(i+w-1, T)$. Contar esos t da $\min(i, w, T-i+1)$. Hay $T-w+1$ ventanas de tamaño w. □

Por lo tanto T_{obs} es una media ponderada de los cosenos, con peso w en el interior y pesos menores en los $w-1$ turnos de cada extremo. La ventana apenas altera la prueba: esta evalúa el *emparejamiento prompt–respuesta*, no el orden secuencial de los turnos.

6.3 Condiciones e interpretación

- **Tamaño.** Con T turnos existen $T!$ permutaciones distintas; para $T < 7$ ese número es menor que $B = 1000$ y el p mínimo alcanzable queda limitado. Se recomienda $T \ge 10$.
- **Deriva temática.** Si el tema cambia lentamente, un emparejamiento barajado puede conservar similitud residual o, al revés, penalizar de más; el nulo no controla ese efecto.
- **Relevancia frente a reflejo.** Cualquier respuesta pertinente se parece más a su prompt que una respuesta ajena; un p pequeño es esperable en diálogos coherentes y no distingue relevancia de «reflejo».
- **Comparaciones múltiples.** Al probar varios diálogos hay que corregir (Holm o Bonferroni).
- **Estatus.** Es un análisis exploratorio; no constituye replicación independiente.

7 Implementación de referencia

La implementación (`pao_pnlio_v1_3.py`, Python 3.10+, dependencias `numpy` y, opcionalmente, `sentence-transformers`) expone la clase `PAOKernel`. Cada turno se agrega con `add_turn(prompt, response, telemetry)`; `analyze()` devuelve similitudes, medias móviles, pendiente y turnos Reflex; `permutation_test()` ejecuta el protocolo de la Sección 6. Los vectores se normalizan siempre en `embed()`, incluso si un modelo inyectado no lo hace, porque la prueba de permutación usa producto punto. El criterio Reflex, en esencia:

```
def moving_average(values, window):
    out = [None] * len(values)
    for i in range(window - 1, len(values)):
        out[i] = fmean(values[i - window + 1 : i + 1])
    return out

# criterio Reflex (dentro de add_turn)
rolling = moving_average(similarities, window_size)
valid = [x for x in rolling if x is not None]
if len(valid) >= k_reflex:
    reflex = all(theta <= x <= upper for x in valid[-k_reflex:])
```

Uso por línea de comandos. Un archivo JSONL con un objeto por línea (campos obligatorios `prompt` y `response`; opcionales `latency_ms`, `prompt_tokens`, `completion_tokens`, `context_depth`, `memory_usage_mb`):

Copyright © 2026 Gonzalo Mauricio De la Rivera Arellano · Licensed under the Apache License, Version 2.0

```
pip install -r requirements-pao-pnlio.txt
python pao_pnlio_v1_3.py --input turnos.jsonl --output resultados.json
```

El modelo por defecto es multilingüe, apropiado para diálogos en español. *Cambiar de modelo de embeddings cambia la escala de las similitudes*; por eso los umbrales 0.75 y 0.90 no son comparables con los del Analyzer v1.2, que usaba `all-MiniLM-L6-v2` (centrado en inglés) y deben recalibrarse.

Versión	Definición de C	Cambio principal
v1.1	$C_t = S_t/t$	Cota $1/t$: Reflex solo posible en $t = 1$ (Proposición 1).
v1.2	media móvil de S_t (ventana 3), Reflex en banda sostenida	Elimina la división por turno; añade línea base aleatoria con p-valor.
v1.3	igual, con C_t indefinido para $t < w$	Normalización forzada en `embed()`, error relativo para la regulación, modelo multilingüe, hipótesis y limitaciones explícitas.

Cuadro 4: Evolución de la definición de coherencia.

8 Verificación de la implementación

El archivo `test_pao_pnlio_v1_3.py` contiene 10 pruebas unitarias que se ejecutan con modelos falsos, sin descargar pesos. Todas pasan. Verifican propiedades matemáticas y de software:

N.º	Propiedad verificada
1	La media móvil deja `None` en las $w - 1$ primeras posiciones.
2	El coseno con un vector de norma cero vale 0.
3	$H = 2$ bits para cuatro símbolos equiprobables.
4	`embed()` devuelve vectores de norma 1 aunque el modelo no normalice.
5	El estadístico de permutación queda en $[-1, 1]$ (escala del coseno).
6	Con $S_t = 0.8$ constante, $w = 3$, $k = 2$, el primer Reflex ocurre en $t = 4 = w + k - 1$ (Proposición 2).
7	Similitudes constantes fuera de la banda (0.95 y 0.40) no activan Reflex.
8	La permutación detecta un emparejamiento perfecto (10 turnos ortogonales; $p < 0.05$ con $B = 500$).
9	El error relativo es comparable entre unidades (5 % en latencia, 2.5 % en memoria).
10	La regulación usa el error relativo y no se aplica automáticamente.

Cuadro 5: Pruebas unitarias de la implementación de referencia.

Estas pruebas acreditan que el código implementa las definiciones de la Sección 4; *no* ponen a prueba las hipótesis H1–H5, que requieren corpus reales. Al ejecutarlas contra una versión previa del archivo, tres fallaban (normalización de vectores en dos pruebas y error relativo), lo que motivó las correcciones descritas en la Sección 7.

9 Limitaciones

1. **Similitud no es coherencia.** El coseno entre embeddings mide cercanía temática o léxica. Una respuesta correcta que cambia de registro puede puntuar bajo; una paráfrasis vacía puede puntuar alto.

2. **Relevancia frente a reflejo.** Una similitud alta es compatible con que el sistema refleje al usuario, pero también con que simplemente responda al tema. El marco no puede distinguirlas sin controles adicionales.
3. **Convenciones de umbral.** $\theta = 0.75$ y $U = 0.90$ son valores por defecto, no resultados empíricos; dependen del modelo de embeddings (H3).
4. **Alcance del nulo.** La prueba de permutación estima el emparejamiento, no el orden; es sensible a la deriva temática y no corrige comparaciones múltiples por sí sola.
5. **Autodescripción.** Un modelo de lenguaje no tiene acceso intrínseco a su latencia o memoria; E_t mide la discrepancia entre lo que dice y lo medido, no una introspección.
6. **Antecedentes inferidos de fotografías.** La Sección 3 se apoya en $n = 7$ lecturas transcritas de capturas; es una inferencia, no una validación.
7. **Sin datos reales.** No se evaluó el marco con corpus ni modelos; las pruebas unitarias usan modelos falsos.
8. **Sin evaluación humana ni revisión externa.** El documento es de una sola autoría.
9. **Regulación no medida.** No se mide si la acción recomendada mejora el desempeño posterior.

10 Trabajo futuro

- Reunir un corpus de diálogos con metadatos (modelo, configuración, fecha) y ejecutar H1–H3.
- Añadir controles: prompts con premisas falsas, comparación entre modelos y evaluación humana a ciegas.
- Calibrar $[\theta, U]$ por modelo de embeddings, por ejemplo a partir de percentiles de la distribución nula.
- Marcar y excluir turnos con error de generación (*timeouts*) antes de calcular S_t.
- Implementar z_t, Rec_t, Q_t y la integración de la entropía; ejecutar H4 y H5.
- Preinscribir el análisis (hipótesis, estadístico y criterios) antes de recolectar los datos.

11 Reproducibilidad y disponibilidad

Código, pruebas y dependencias se distribuyen con el repositorio del proyecto (`https://github.com/godear6959-creator/PNLIO-Framework`). La prueba de permutación usa una semilla fija (42) y registra en su salida el estadístico, el número de permutaciones y el p-valor. Para reproducir los resultados deben fijarse el modelo de embeddings (nombre y versión), los parámetros w, k, θ, U, B y la semilla. Las 10 pruebas unitarias se ejecutan con `python -m pytest`.

12 Licencia y autoría

Este documento y el código fuente asociado se distribuyen bajo la **Apache License, Version 2.0** (`http://www.apache.org/licenses/LICENSE-2.0`). Copyright © 2026 Gonzalo Mauricio De la Rivera Arellano. Al redistribuir o modificar la obra deben conservarse el aviso de copyright, una

copia de la licencia y la indicación de los cambios realizados, según los términos de la licencia; el texto completo acompaña al repositorio en el archivo `LICENSE`.

Referencias

[1] De la Rivera Arellano, G. M. (2026). *PNLIO Framework: Ollama Local Kernel, Symbiotic Memory and RCR Metric Specifications*. Repositorio del proyecto PNLIO.

[2] De la Rivera Arellano, G. M. (2026). *PNLIO Coherence Analyzer v1.2: especificación y guía explicativa*. Documento de trabajo, proyecto PNLIO.

[3] Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3), 379–423.

[4] Vaswani, A., et al. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998–6008.

[5] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *Proceedings of EMNLP-IJCNLP 2019*, 3982–3992.

[6] Reimers, N., & Gurevych, I. (2020). Making monolingual sentence embeddings multilingual using knowledge distillation. *Proceedings of EMNLP 2020*, 4512–4525.

[7] Wang, W., et al. (2020). MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. *Advances in Neural Information Processing Systems*, 33.

[8] Phipson, B., & Smyth, G. K. (2010). Permutation p-values should never be zero: calculating exact p-values when permutations are randomly drawn. *Statistical Applications in Genetics and Molecular Biology*, 9(1), Article 39.

[9] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2.ª ed.). MIT Press.